

Project Executable Files

Date	05 July 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	YES
2	README / Setup Guide (instructions to run the project)	YES
3	requirements.txt / package.json / dependency file	YES
4	Database schema / seed files (if applicable)	Yes (Handled automatically by SQLAlchemy schema initialization on app startup)
5	Environment configuration file (.env.example or similar)	NA (Local configuration parameters are integrated natively)
6	Deployed application URL (if hosted)	Yes (Publicly accessible via active Pinggy tunnel)
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA

9	Test files and test results	Yes (Includes locustfile.py and Locust performance metrics)
10	Demo video or walkthrough (if required)	YES

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```
Smart Lender/
├── Dataset/
│   ├── loan_prediction.csv
│   ├── loan_prediction(1).csv
│   └── loan_prediction.xlsx
├── static/
│   ├── css/
│   │   └── style.css
│   └── images/
│       └── plots/
│           ├── bivariate.png
│           ├── confusion_matrix.png
│           ├── education_vs_loan_status.png
│           ├── gender_count.png
│           ├── income_distribution.png
│           ├── loan_status_count.png
│           ├── model_comparison.png
│           ├── multivariate.png
│           ├── univariate_count.png
│           ├── univariate_dist.png
│           └── xgb_cv_scores.png
├── templates/
│   ├── layout.html
│   ├── home.html
│   ├── predict.html
│   ├── submit.html
│   └── history.html
├── app.py
├── application.py
├── models.py
├── train_models.py
├── tune_xgb.py
├── locustfile.py
├── vercel.json
├── .gitignore
├── requirements.txt
├── smart_lender.db
├── rdf.pkl
├── scale1.pkl
└── metrics.pkl
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://kvnvk.pythonanywhere.com/
Login Credentials (Demo)	Not Required (Public Application)
Platform / Hosting Provider	PythonAnywhere
Repository Link	https://github.com/kvnvk9955-stack/Smart-Lender
Demo Video Link	https://drive.google.com/file/d/1O5voAYSTYKgEXtLIMTMfywglh7nz5JMN/view?usp=sharing

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites:

Python 3.9 or higher installed.

Python package manager (pip) installed.

Standard modern web browser.

Step-by-step local launch instructions:

- **Step 1 (Clone the repository):**

```
git clone https://github.com/kvnvk9955-stack/Smart-Lender.git
```

```
cd Smart-Lender
```

- **Step 2 (Install dependencies):**

```
pip install flask flask_sqlalchemy pandas numpy scikit-learn xgboost matplotlib
```

```
seaborn
```

- **Step 3 (Configure models / optional):**

Verify the model assets exist in the root folder (rdf.pkl, scale1.pkl). If you wish to retrain from the raw dataset and re-plot the EDA, run: `python train_models.py` and `python tune_xgb.py`

- **Step 4 (Database migration/initialization):**

The SQLite database configuration automatically creates schemas and registers default model metadata on app start. No manual database setup is required.

- **Step 5 (Start the application):**

```
python app.py
```

- **Step 6 (Open browser):**

Open your browser and navigate to: `http://localhost:5001`

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	SQLite database file locking during highly concurrent prediction writes.	Workaround: Thread-safe SQLAlchemy scoped sessions are implemented. For enterprise deployments, migrating the URI connection to PostgreSQL is recommended.
2	Python version compatibility mismatches when importing the serialized Pickle (.pkl) models.	Workaround: If a pickle compatibility error triggers, simply re-run <code>python train_models.py</code> to regenerate the pickles using your local python environment version.
3	Inference cold start delay during the first prediction	Workaround: Deployed a global

execution.

model loader routine
(`load_prediction_model()`)
executing at server start rather
than loading the pickle inside route
requests.